Implement A* Search & AO* Search algorithms.

Difference Between A* and AO* Algorithm

Search algorithms are widely used in artificial intelligence and graph traversal nowadays. ***A (A-star)**** and ***AO (And-Or-star)**** are informed search algorithms that aim to find optimal solutions, but they are designed to work in different contexts and solve distinct types of problems.

In this article, we will focus on the differences between those algorithms based on different criteria.

A* Algorithm

A*(A-star) algorithm is an **informed search algorithm** leverages both the known cost to reach a point and a **heuristic**—an estimate of the remaining distance to the goal. This combination makes A* a powerful tool for finding **optimal paths** while minimizing unnecessary exploration.

How it works:

- A* uses **g(n)** (the actual cost from the start node to the current node) and **h(n)** (the heuristic estimate from the current node to the goal).
- It selects the path that minimizes **f(n) = g(n) + h(n)**.
- A* ensures optimal solutions in scenarios with well-defined paths, such as **robot navigation** and **game development**.

*A (A-star) algorithm** is a widely used **path-finding algorithm** in computer science, known for its efficiency and effectiveness in **route planning** for applications such as **maps, robotics, and games**.

AO* Algorithm

The ***AO* (And-Or-star) algorithm*** is a *variant of the A algorithm**, designed to be **more adaptive and flexible**, particularly in **dynamic environments**. AO* builds on A*'s capabilities by allowing **on-the-fly adjustments** whenever it encounters changes, making it an ideal choice for scenarios where conditions are uncertain or constantly evolving.

How it works:

- AO* explores a search graph with **AND** and **OR nodes**.
- At AND nodes, all child nodes must be solved to proceed.

- At OR nodes, only one child node needs to be solved to continue.
- This algorithm is useful for tasks such as **planning**, **game strategy development**, and **automated reasoning**.

Difference Between A* and AO* Algorithm

Aspect	***A Algorithm****	***AO Algorithm****
Adaptability to Changing Environments	Not designed for handling changes in the environment.	Specifically designed to adapt to changes without initiating a new search.
OR-AND Operation Combination	Uses the AND operation, focusing on one path at a time.	Uses both OR and AND operations, exploring multiple paths simultaneously.
Resource Utilization	Generally more resource-efficient, explores fewer nodes.	May explore more nodes due to adaptability, potentially requiring more computational resources.
Planning for Uncertainty	Less suited for high uncertainty or frequent environmental changes.	Excels in situations with uncertainty, quickly adjusting plans in response to new information.
Search Restart Requirement	Requires a complete restart of the search after an environmental change.	Eliminates the need for a full restart, saving time and computational resources when changes occur.
Scenario Suitability	Well-suited for static environments with consistent node costs.	Particularly beneficial in dynamic environments where conditions or costs may change over time.

Aspect	*A Algorithm**	*AO Algorithm**
Robustness to Changes	May struggle in environments subject to frequent alterations.	Handles changes seamlessly, ensuring that plans remain effective even as the environment evolves.
Real-time Applications	Effective in static scenarios with minimal changes.	Highly suitable for real-time applications with dynamic elements.
Memory Usage	Uses less memory due to fewer nodes being explored.	May use more memory to store additional information about explored paths and alternative solutions.
Consistency of Heuristic	Requires a consistent heuristic for optimal results.	Does not strictly require a consistent heuristic, offering more flexibility in heuristic choice.

Which Algorithm Should You Use?

- Use **A*** if you are working with pathfinding or graph traversal problems, especially when you need the shortest path between two points.
- Use **AO*** if the problem involves dependencies between sub-goals, such as in game trees, planning systems, or decision-making scenarios.

A* Search Algorithm (Idea)

🔹 **What it does:**

A* is used to find the **shortest path** from a start node to a goal node.

🔹 **Key formula:**

It chooses the best node using:

f(n)=g(n)+h(n)f(n) = g(n) + h(n)f(n)=g(n)+h(n)

f(n)=g(n)+h(n)f(n)=g(n)+h(n)f(n)=g(n)+h(n)

🔹 **Meaning:**

- **g(n)** → actual cost from start to current node
- **h(n)** → estimated cost to goal (heuristic)
- **f(n)** → total estimated cost

👉 It always expands the node with **minimum f(n)**

Simple analogy:

Think of **Google Maps**:

- Distance already traveled → g(n)
- Remaining distance estimate → h(n)
- Best route = minimum total distance

Think like this (very simple):

You are going from **Home → College**

At some point, you have 2 routes:

Route 1:

- Already traveled = 5 km
- Remaining (estimated) = 5 km
  👉 Total = **10 km**

Route 2:

- Already traveled = 3 km
- Remaining (estimated) = 10 km
  👉 Total = **13 km**

🔥 **Which is better?**

👉 **Route 1 (10 km)** is better than **Route 2 (13 km)**

This is exactly what A* does:

It calculates:

- distance already traveled → g(n)
- estimated distance left → h(n)

Then adds them:

$f(n)=g(n)+h(n)f(n) = g(n) + h(n)f(n)=g(n)+h(n)$

👉 The path with **smallest f(n)** is chosen.

AO* Search Algorithm (Idea)

🔹 **What it does:**

AO* is used for **AND-OR graphs** (decision problems).

🔹 **Key idea:**

- Sometimes you need **ALL children (AND)**
- Sometimes only **ONE option (OR)**

🔹 **How it works:**

- It finds the **minimum cost solution**
- Uses:

👉 **Cost = sum(child cost + edge weight)**
👉 Chooses **minimum among all options**

Simple analogy:

Think of planning:

- "To pass exam":
    - Study **AND** practice (both needed)
- Or choose:
    - Option 1: Assignment
    - Option 2: Project

AO* finds the **cheapest valid plan**

A

/ \

(B AND C) OR (D)

Costs:

- A → B = 1, B = 2
- A → C = 1, C = 2
  👉 AND path cost = **1+2 + 1+2 = 6**
- A → D = 1, D = 3
  👉 OR path cost = **4**

✅ **Best choice:**

👉 AO* selects **minimum cost**

So:

- AND path = 6
- OR path = 4

👉 Chooses **A → D** ✅

🧠 **How AO* works (logic)**

1. Start from root
2. Explore all options (AND / OR)
3. Compute cost:
   - AND → sum of all children
   - OR → choose minimum
4. Update best path
5. Repeat until optimal solution found

```
from heapq import *

# -------- A* --------
def astar(g, s, t, h):
    q = [(h[s], 0, s, [])]; v = set()
    while q:
        f, c, n, p = heappop(q)
        if n in v: continue
        p = p + [n]; v.add(n)
        if n == t: return p, c
        for x, w in g.get(n, []):
            heappush(q, (c + w + h[x], c + w, x, p))

# -------- AO* --------
def ao(n, g, h, sol):
    if n not in g:
        sol[n] = (h[n], [n])
        return h[n]
```

```
    best_cost, best_path = float('inf'), []

    for op in g[n]:
        cost = 0; path = [n]
        for x, w in op:
            cost += ao(x, g, h, sol) + w
            path += sol[x][1]

        if cost < best_cost:
            best_cost, best_path = cost, path

    sol[n] = (best_cost, best_path)
    return best_cost

# -------- Example --------
g1 = {'A':[('B',1),('C',3)], 'B':[('D',3),('E',1)], 'E':[('G',2)]}
h1 = {'A':5,'B':3,'C':4,'D':6,'E':2,'G':0}

print("A*:", astar(g1,'A','G',h1))

g2 = {'A':[[('B',1),('C',1)],[('D',1)]], 'B':[[('E',1)],[('F',1)]]}
h2 = {'A':5,'B':3,'C':4,'D':2,'E':6,'F':4}

solution = {}
ao('A', g2, h2, solution)
print("\nSolution Graph:")
for node, (cost, path) in solution.items():
    print(f"Node: {node}, Cost: {cost}, Path: {path}")
```

Output

A*: (['A', 'B', 'E', 'G'], 4)

Solution Graph:

Node: E, Cost: 6, Path: ['E']

Node: F, Cost: 4, Path: ['F']

Node: B, Cost: 5, Path: ['B', 'F']

Node: C, Cost: 4, Path: ['C']

Node: D, Cost: 2, Path: ['D']

Node: A, Cost: 3, Path: ['A', 'D']

Run completed in 3770.1000000089407ms